

CertManager (Aplicativo N1) — Necesidades de instalación

Documento: Solicitud de necesidades de instalación y alcance. **Versión:** 1.0

Este documento lista TODO lo que el equipo de infraestructura debe aprovisionar para instalar y operar CertManager.

1. Alcance del aplicativo

Aspecto	Detalle
Tipo	Aplicación web (server-rendered) + API REST
Lenguaje / runtime	Python 3.11+ (probado en 3.13)
Servidor de aplicación	Gunicorn (WSGI) detrás de NGINX
Base de datos	MySQL 8 — EXTERNA, la provee Claro (no se instala con el app)
Tareas programadas	APScheduler en-proceso (<code>manage.py run_scheduler</code>)
Estáticos	Servidos por la propia app (WhiteNoise), no requiere CDN
Autenticación	Local (usuario/clave) y/o LDAP/AD corporativo
¿Consume URLs externas?	Sí — ver sección 4 (es su función: conectarse a los hosts a monitorear).

2. Requisitos del servidor (instalación local / como servicio)

2.1 Sistema operativo y dimensionamiento

- **SO:** Linux (RHEL/Rocky 8/9, o Debian/Ubuntu LTS). 64-bit.
- **CPU:** 2 vCPU (mínimo) / 4 vCPU (recomendado).
- **RAM:** 2 GB (mínimo) / 4 GB (recomendado).
- **Disco:** 10 GB libres (app + logs + estáticos). Logs rotan (10 MB × 10).
- **Usuario de servicio:** dedicado, sin shell de login (p.ej. `certmanager`).

2.2 Software base a tener instalado

Software	Uso	Obligatorio
Python 3.11+ con <code>venv</code> y <code>pip</code>	Runtime de la app	Sí
Compilador C + headers (<code>gcc</code> , <code>python3-devel</code> / <code>python3-dev</code>)	Compilar <code>mysqlclient</code> , <code>cryptography</code>	Sí

Software	Uso	Obligatorio
Cliente/headers MySQL (<code>mysql-devel</code> / <code>default-libmysqlclient-dev</code> , <code>pkg-config</code>)	Driver MySQL	Sí
Librería runtime MySQL (<code>libmariadb</code> / <code>mysql-libs</code>)	Driver MySQL en ejecución	Sí
NGINX	Reverse proxy + terminación TLS	Sí (prod)
<code>gettext</code> (<code>msgfmt</code>)	Compilar traducciones i18n (opcional: el <code>.mo</code> ya viene compilado)	Opcional
Node.js 20 + npm	Compilar el CSS (Tailwind) si no se entrega ya compilado	Opcional (ver 2.3)
<code>curl</code>	Healthcheck	Recomendado

Paquetes Python: ver `requirements/prod.txt` (Django, DRF, cryptography, APScheduler, requests, ldap3, pyotp, reportlab, openpyxl, mysqlclient, gunicorn, whitenoise, y **opcionalmente** obsforge desde índice privado).

2.3 Sobre el CSS (Tailwind)

El archivo `static/css/forge.css` se genera con `npm run build:css`. Dos opciones:

- **(A)** Instalar Node.js 20 en el servidor y compilarlo durante la instalación (el script lo hace si detecta `npm`).
- **(B)** Entregar el `forge.css` ya compilado (build en otra máquina) y copiarlo a `static/css/forge.css` → **el servidor no necesita Node**.

3. Base de datos (EXTERNA — solicitar a Claro)

CertManager **no instala** la base de datos. Se debe solicitar:

Parámetro	Valor a solicitar
Motor	MySQL 8.0
Charset / Collation	<code>utf8mb4</code> / <code>utf8mb4_unicode_ci</code>
Base de datos	p.ej. <code>certmanager</code>
Usuario / contraseña	con permisos <code>ALL</code> sobre esa BD (incluye crear/alterar tablas para migraciones)
Host / puerto	FQDN o IP del MySQL + 3306
Conectividad	El servidor de app debe alcanzar el MySQL por TCP 3306
<code>sql_mode</code>	la app fuerza <code>STRICT_TRANS_TABLES</code> por conexión

Configuración resultante (en el `.env`, valores **por separado**):

```
DB_NAME=certmanager
DB_USER=certmanager
DB_PASSWORD=<contraseña de Claro>
DB_HOST=<host/IP del MySQL>
DB_PORT=3306
```

4. Conectividad SALIENTE (egress) — reglas de firewall

La función principal del aplicativo es conectarse a los hosts a monitorear, así que **requiere salida**. Solicitar apertura de:

Destino	Puerto	Protocolo	Propósito	¿Obligatorio?
Hosts/dominios a monitorear	443 (y los puertos que se configuren por cert)	TLS	Chequeo de certificados (núcleo)	Sí
Servidor MySQL (Claro)	3306	TCP	Base de datos	Sí
Servidor SMTP	587 / 465 / 25	SMTP(S)	Correo de alertas y reportes	Si se usa correo
LDAP / Active Directory	389 / 636	LDAP / LDAPS	Autenticación corporativa	Si se usa LDAP
Gateway SMS	21 (+ rango pasivo FTP)	FTP	Notificaciones SMS	Si se usa SMS
Webhooks (Teams/Slack/otros)	443	HTTPS	Notificaciones a canales	Si se usan webhooks
<code>cdn.jsdelivr.net</code>	443	HTTPS	Solo la UI de Swagger en <code>/api/docs/</code>	Opcional (ver nota)
Loki (si se usa push HTTP)	según despliegue	HTTP(S)	Observabilidad	Opcional

Notas de egress:

- La app **no** usa CDNs para la interfaz principal (htmx, fuentes, CSS, JS se sirven localmente vía WhiteNoise). La **única** dependencia de CDN es la página de documentación de la API (`/api/docs/` , Swagger UI desde `cdn.jsdelivr.net`). Si el entorno es cerrado, esa página no renderiza el visor pero la API funciona igual; se puede empaquetar el visor localmente si se requiere.
- Por defecto la observabilidad (`obsforge`) escribe a **stdout** (sin red); un agente externo (Promtail) lo recoge. No abre conexiones salientes salvo que se configure push HTTP a Loki.
- Anti-SSRF:** por defecto el chequeo rechaza hosts que resuelvan a IPs privadas/loopback/metadata. Para monitorear **hosts internos** de Claro hay que habilitarlo explícitamente (parámetro de configuración) — indicarlo al equipo.

5. Conectividad de INSTALACIÓN (build-time)

Durante la instalación, el servidor debe **alcanzar** (o tener un mirror interno):

Recurso	Destino	Puerto	Para
Índice PyPI	<code>pypi.org</code> / mirror interno	443	<code>pip install -r requirements/*.txt</code>
Índice privado (obsforge)	repositorio interno Claro	443	Solo si se instala <code>obsforge</code> (opcional)
Registro npm	<code>registry.npmjs.org</code> / mirror	443	<code>npm ci</code> para compilar CSS (opcional — ver 2.3)
Repos del SO	mirror apt/yum interno	443/80	Paquetes de sistema (gcc, libmysqlclient, nginx)

Si el servidor **no** tiene salida a internet: usar mirrors internos de PyPI/npm y entregar el `forge.css` ya compilado. `obsforge` es opcional (si el índice privado no está disponible, se omite y la app funciona igual).

6. Conectividad ENTRANTE (ingress)

Origen	Puerto	Protocolo	Destino
Usuarios / consumidores API	443	HTTPS	NGINX
(interno) NGINX → Gunicorn	8000 (loopback)	HTTP	Gunicorn (solo <code>127.0.0.1</code>)
Balanceador / monitoreo	443	HTTPS	<code>GET /health/</code> (200 si BD OK, 503 si no; sin auth)

7. Certificado TLS del propio aplicativo

NGINX termina TLS. Solicitar/instalar:

- **Certificado del FQDN** del aplicativo (p.ej. `certmanager.claro.com.do`) + su clave privada y cadena intermedia.
- Rutas a configurar en NGINX (las usa el script `install_server.sh`).

8. Variables de entorno

Todas viven en el `.env` (ver `CLARO_NECESIDAD/.env.example` y el documento `03_cambios_para_produccion.md`). Las **obligatorias** en producción:

Variable	Descripción
<code>DJANGO_SETTINGS_MODULE</code>	<code>config.settings.prod</code>
<code>DJANGO_SECRET_KEY</code>	Clave secreta (generar, ver doc 03)
<code>ALLOWED_HOSTS</code>	FQDN(s) del aplicativo

Variable	Descripción
DB_NAME / DB_USER / DB_PASSWORD / DB_HOST / DB_PORT	MySQL externa (de Claro), valores por separado
CSRF_TRUSTED_ORIGINS	https://<fqdn>

Opcionales: EMAIL_* (SMTP), LOG_DIR, SECURE_*, OBSFORGE_*, CF_OWNER_.